1. Problem Title & Link

Title: **LeetCode 452: Minimum Number of Arrows to Burst Balloons**

Link: https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/

2. Problem Statement (Short Summary)

You are given an array points where:

points[i] = [start, end]

Each pair represents a **horizontal balloon interval**.

If you shoot an arrow at position x, it will burst **all balloons whose interval contains x**.

Goal:

Find the **minimum number of arrows required to burst all balloons**.

3. Examples (Input → Output)

Example 1

Input

points = [[10,16],[2,8],[1,6],[7,12]]

Output

2

Explanation

Arrows shot at:

x = 6

x = 11

First arrow bursts:

[2,8], [1,6]

Second arrow bursts:

[7,12], [10,16]

Example 2

Input

points = [[1,2],[3,4],[5,6],[7,8]]

Output

4

Explanation

No overlapping intervals, so each balloon needs its own arrow.

Example 3

Input

points = [[1,2],[2,3],[3,4],[4,5]]

Output

2

Explanation

Arrow at 2 bursts first two balloons.

Arrow at 4 bursts remaining balloons.

4. Constraints

1 ≤ points.length ≤ 100000

points[i].length == 2

-2^31 ≤ start < end ≤ 2^31 - 1

Observations:

- Large input size
- We must avoid $O(n^2)$ solutions
- Problem resembles **interval merging**

5. Core Concept (Pattern / Topic)

Greedy + Sorting (Interval Scheduling)

Idea:

Shoot arrows where **maximum balloons overlap**.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

Try every possible arrow position.

Check which balloons burst.

This leads to **$O(n^2)$** complexity.

Not feasible.

Step 2 — Key Observation

If two balloons overlap:

[1,6]

[2,8]

An arrow anywhere in:

[2,6]

will burst both balloons.

So we should shoot the arrow **at the end of the overlapping interval**.

Step 3 — Greedy Strategy

1. Sort balloons by **end coordinate**
2. Shoot arrow at the **end of the first balloon**
3. Continue scanning:
    - If next balloon starts after arrow → need new arrow
    - Otherwise → same arrow bursts it

7. Visual / Intuition Diagram (ASCII)

Example

[1,6]

[2,8]

[7,12]

[10,16]

After sorting by end:

[1,6]

[2,8]

[7,12]

[10,16]

Arrow at 6 bursts:

[1,6]

[2,8]

Next arrow at 12 bursts:

[7,12]

[10,16]

Total arrows = **2**

8. Pseudocode (Language Independent)

```
sort balloons by end coordinate

arrows = 1
arrow_position = end of first balloon

for each balloon:

  if balloon.start > arrow_position:

    arrows += 1
```

```
        arrow_position = balloon.end

return arrows
```

9. Code Implementation

Python

```
class Solution:
    def findMinArrowShots(self, points):

        points.sort(key=lambda x: x[1])

        arrows = 1
        arrow_pos = points[0][1]

        for start, end in points:

            if start > arrow_pos:
                arrows += 1
                arrow_pos = end

        return arrows
```

Java

```
class Solution {
    public int findMinArrowShots(int[][] points) {

        Arrays.sort(points, (a,b) -> Integer.compare(a[1], b[1]));

        int arrows = 1;
        int arrowPos = points[0][1];

        for(int i = 1; i < points.length; i++){

            if(points[i][0] > arrowPos){

                arrows++;
                arrowPos = points[i][1];
```

```
        }
    }

    return arrows;
  }
}
```

10. Time & Space Complexity

Time Complexity

Sorting:

O(n log n)

Traversal:

O(n)

Total:

O(n log n)

Space Complexity

O(1)

(ignoring sorting overhead)

11. Common Mistakes / Edge Cases

1. Sorting by **start instead of end**
2. Using >= instead of >
3. Forgetting to initialize arrow count
4. Missing interval overlap cases

12. Detailed Dry Run

Example

points = [[10,16],[2,8],[1,6],[7,12]]

Sorted by end

[1,6]

[2,8]

[7,12]

[10,16]

Interval	Arrow Position	Action	Arrows

[1,6]	6	shoot arrow	1
[2,8]	6	overlap	1
[7,12]	7>6 new arrow	2	
[10,16]	overlap	2	

Final answer

2

13. Common Use Cases (Real-Life / Interview)

Interval greedy problems appear in:

- Meeting room scheduling
- CPU job scheduling
- Resource allocation
- Network packet batching
- Event timeline optimization

14. Common Traps (Important!)

1. Sorting by wrong column
2. Shooting arrow at start instead of end
3. Using wrong overlap condition
4. Forgetting intervals touching at boundary

15. Builds To (Related LeetCode Problems)

Interval greedy progression:

LC 252 Meeting Rooms

LC 435 Non-overlapping Intervals

LC 452 Minimum Number of Arrows

LC 56 Merge Intervals

LC 253 Meeting Rooms II

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	$O(n^2)$	Too slow

Greedy + Sorting	O(n log n)	Optimal

17. Why This Solution Works

By always shooting the arrow at the **earliest possible end point**, we maximize the number of balloons that can be burst by the same arrow.

This greedy choice ensures the **minimum arrows are used**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Minimum arrows if balloons are **2D circles**
2. Maximum balloons burst with **k arrows**
3. Merge overlapping balloons first
4. Online streaming balloon intervals